RETRIEVAL-AUGMENTED GENERATION FOR HIGH-PERFORMANCE COMPUTING SUPPORT

A DISSERTATION SUBMITTED TO THE UNIVERSITY OF MANCHESTER
FOR THE DEGREE OF BACHELOR OF SCIENCE
IN THE FACULTY OF SCIENCE AND ENGINEERING

2026

Student id: 11006713

Department of Computer Science

Contents

List of Abbreviations **8**

Abstract **9**

Declaration **10**

Copyright **11**

Acknowledgements **12**

1 Introduction **13**

1.1 Project Context and Motivation 13
1.2 Problem Definition and Scope 14
1.3 Research Questions and Hypotheses 14
1.4 Aim, Objectives, and Success Criteria 15
1.5 Overview of Evaluation Strategy 16
1.6 Report Structure 17

2 Background and Related Work **18**

2.1 HPC Support as an Information Retrieval and Question Answering Problem 18
2.2 Retrieval-Augmented Generation 20
2.3 Heterogeneous Support Corpora: Documentation and Tickets 22
2.4 Authority, Freshness, and Validity in Retrieval 24
2.5 Evaluation of RAG Systems 26
2.6 Gap Analysis and Research Positioning 27

3 Problem Formulation and Requirements **29**

3.1 Task Definition 29

3.2 Data Sources and Knowledge Characteristics 30
3.3 Benchmark and Query Taxonomy 32
3.4 Requirements and Constraints 34
3.5 Success Criteria and Requirement-to-Evaluation Mapping 35
3.6 Ethical, Privacy, and Data-Governance Considerations 37

4 System Design and Implementation 39
4.1 System Overview 39
4.2 Baseline Systems 41
4.3 Corpus Ingestion and Representation 42
4.4 Authority Registry and Metadata Extraction 43
4.5 Validity-Aware Reranking Design 45
4.6 Prompt and Generation Design 46
4.7 API, Tooling, and Observability 47
4.8 User Interface and Interaction Design 48
4.9 Verification and Software Testing 48
4.10 Design Trade-offs 49

5 Experimental Methodology 51
5.1 Evaluation Goals 52
5.2 Development and Test Split Discipline 52
5.3 Benchmark Construction and Characterisation 52
5.4 Frozen Backbone Decisions 52
5.5 Metric Hierarchy and Decision Rules 52
5.6 Development-Phase Tuning Protocol 52
5.6.1 Generator Selection 52
5.6.2 Chunking Tuning 52
5.6.3 Retrieval-Mode Pilot 52
5.6.4 Shared Retrieval Budget Tuning 52
5.6.5 Validity-Reranker Tuning 52
5.7 Final Test-Set Experiments 52
5.7.1 Source Ablation 52
5.7.2 Validity-Signal Ablation 52
5.7.3 External-Document Scope Ablation 52
5.7.4 Tuned-Finalist Comparison 52
5.8 Manual Evaluation Protocol 52

5.9 Error Analysis Protocol 52
5.10 Reproducibility and Stability Controls 52

6 Results **53**
6.1 Development-Phase Selection Results 53
6.2 Overview of Main Test-Set Results 53
6.3 Controlled Ablation Results 53
6.3.1 Source Ablation 53
6.3.2 Validity-Signal Ablation 53
6.3.3 External-Document Scope Ablation 53
6.4 Practical Comparison Results 53
6.4.1 Tuned-Finalist Comparison 53
6.5 Manual Evaluation Results 53
6.6 Error Analysis Results 53
6.7 Stability and Robustness Results 53

7 Discussion and Critical Appraisal **54**
7.1 Answers to the Research Questions 54
7.2 What Improved and Why 54
7.3 Comparison with Prior Work 54
7.4 Where the System Still Fails 54
7.5 Threats to Validity 54
7.6 Review of the Project Plan 54

8 Conclusion and Future Work **55**
8.1 Summary of Contributions 55
8.2 Main Findings Against Objectives 55
8.3 Limitations 55
8.4 Future Work 55

A Approved External Documentation Sources **61**

B Benchmark Taxonomy Label Definitions **63**

C Authority Metadata Fields **65**

Word Count: 0

List of Tables

2.1 The retrieval-control strategies most relevant to validity-aware support retrieval. 21
2.2 Comparison of the evidential roles, risks, and retrieval implications of the three source types in the Cambridge Service for Data Driven Discovery (CSD3) support setting. 22

3.1 Operational summary of the three source classes included in the project corpus. Counts and date bounds refer to the frozen corpus snapshot used for parameter tuning and final evaluation. The full external source list is given in Appendix A. 31
3.2 Functional requirements, quality requirements, and scope constraints derived from the project task definition and benchmark design. 34
3.3 Success-criteria and requirement-to-evaluation mapping for the project. 36

4.1 Summary of the authority registry. 44
4.2 Summary of the Polaris verification strategy. 49

A.1 Approved external documentation source families included in the frozen external documentation snapshot used for parameter tuning and final evaluation. 61

B.1 Benchmark taxonomy label definitions and allowed values. 63

C.1 Authority metadata fields attached during ingestion. 65

List of Figures

1.1 Annual number of resolved CSD3 helpdesk tickets from 2017 to 2025. Data sourced from RCS support ticket records. 14

4.1 High-level architecture of Polaris. 40
4.2 Runtime query flow in Polaris. 41
4.3 Validity-aware reranking in Polaris. 46

List of Abbreviations

RAG	Retrieval-Augmented Generation
LLM	Large Language Model
LM	Language Model
HPC	High-Performance Computing
CSD3	Cambridge Service for Data Driven Discovery
RCS	Research Computing Services
EPSRC	Engineering and Physical Sciences Research Council
QA	Question Answering
IR	Information Retrieval
RGB	Retrieval-Augmented Generation Benchmark
RRF	Reciprocal Rank Fusion

Abstract

RETRIEVAL-AUGMENTED GENERATION FOR HIGH-PERFORMANCE COMPUTING SUPPORT
Adam Calleja
A dissertation submitted to The University of Manchester
for the degree of Bachelor of Science, 2026

Declaration

No portion of the work referred to in this dissertation has been submitted in support of an application for another degree or qualification of this or any other university or other institute of learning.

Copyright

i. The author of this thesis (including any appendices and/or schedules to this thesis) owns certain copyright or related rights in it (the "Copyright") and s/he has given The University of Manchester certain rights to use such Copyright, including for administrative purposes.

ii. Copies of this thesis, either in full or in extracts and whether in hard or electronic copy, may be made **only** in accordance with the Copyright, Designs and Patents Act 1988 (as amended) and regulations issued under it or, where appropriate, in accordance with licensing agreements which the University has from time to time. This page must form part of any such copies made.

iii. The ownership of certain Copyright, patents, designs, trade marks and other intellectual property (the "Intellectual Property") and any reproductions of copyright works in the thesis, for example graphs and tables ("Reproductions"), which may be described in this thesis, may not be owned by the author and may be owned by third parties. Such Intellectual Property and Reproductions cannot and must not be made available for use without the prior written permission of the owner(s) of the relevant Intellectual Property and/or Reproductions.

iv. Further information on the conditions under which disclosure, publication and commercialisation of this thesis, the Copyright and any Intellectual Property and/or Reproductions described in it may take place is available in the University IP Policy (see `http://documents.manchester.ac.uk/DocuInfo.aspx?DocID=24420`), in any relevant Thesis restriction declarations deposited in the University Library, The University Library's regulations (see `http://www.library.manchester.ac.uk/about/regulations/`) and in The University's policy on presentation of Theses

Acknowledgements

Chapter 1

Introduction

1.1 Project Context and Motivation

The CSD3 is an Engineering and Physical Sciences Research Council (EPSRC) Tier-2 national High-Performance Computing (HPC) service hosted by the University of Cambridge Research Computing Services (RCS), supporting computationally intensive research across a wide range of scientific disciplines [1]. Effective use of such systems requires users to be able to follow and understand detailed documentation. These resources are often scattered across multiple sources, ranging from local CSD3 documentation [1] to external resources such as package documentation, creating a barrier for new users [2].

As a result of this complexity, users frequently turn to the RCS helpdesk for support. Figure 1.1 shows that there has been a steady increase in the yearly number of resolved CSD3 helpdesk tickets from 2017 to 2025. This growth indicates not only sustained support demand, but also an expanding archive of operational support knowledge that could be reused to improve user support [3].

Recent work has shown that Retrieval-Augmented Generation (RAG) is a promising approach for technical support Question Answering (QA) because it combines language generation with access to external evidence [2, 4]. However, in heterogeneous support environments, answer quality depends not only on topical relevance, but also on whether the retrieved evidence is authoritative, current, and applicable to the local system state [5, 6]. In the CSD3 setting, this matters because local documentation, external package documentation, and historical support tickets play different evidential roles. The central challenge is therefore to retrieve evidence that is not just similar to the query, but suitable for the current CSD3 context.

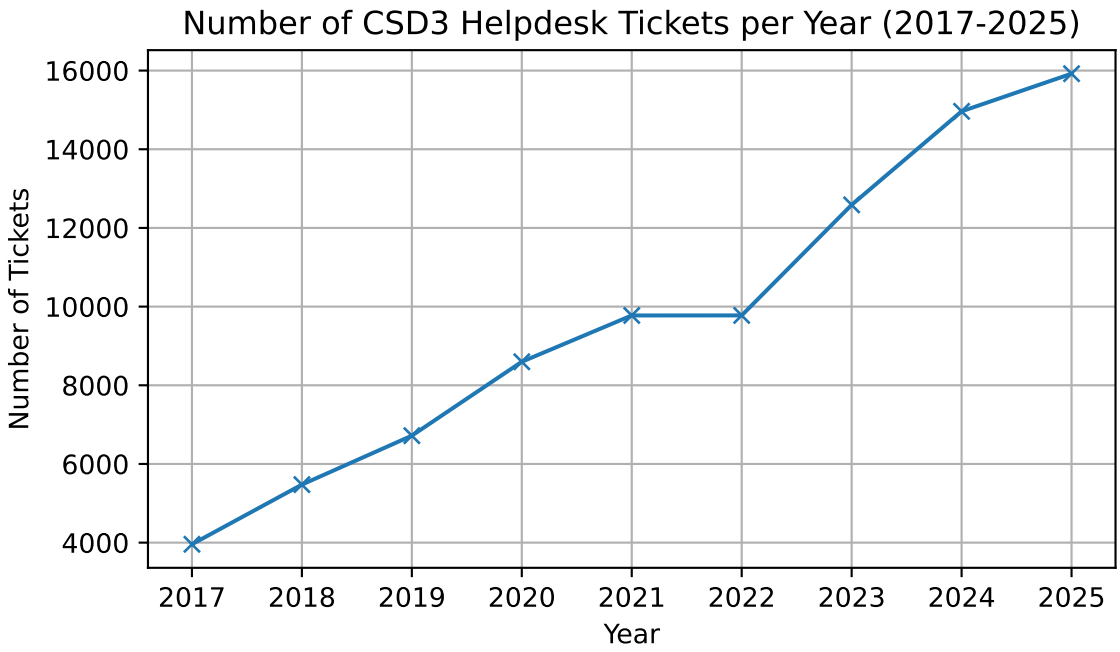

Figure 1.1: Annual number of resolved CSD3 helpdesk tickets from 2017 to 2025. Data sourced from RCS support ticket records.

1.2 Problem Definition and Scope

This dissertation studies evidence-grounded first-response assistance for CSD3 support queries within a supervised internal workflow. The system is limited to single-hop questions, that is, queries answerable in one retrieval step from text-accessible evidence drawn from local documentation, selected external documentation, and visible ticket content. It does not attempt autonomous resolution and excludes attachment-dependent or multimodal evidence such as screenshots and log files. These boundaries keep the task operationally realistic while allowing later experiments to isolate the effect of heterogeneous-source retrieval and validity-aware reranking.

1.3 Research Questions and Hypotheses

To investigate the problems introduced by the heterogeneous evidence base, this project addresses the following research questions (RQ), along with the associated hypotheses (H):

- **RQ1:** Does combining local CSD3 documentation with historical support-ticket

evidence improve the factual correctness of answers, relative to single-source retrieval?

  - **H1:** Combined retrieval will outperform documentation-only and ticket-only baselines in factual correctness, particularly for case-specific troubleshooting queries for which local documentation and historical resolutions provide complementary evidence.

- **RQ2:** Does validity-aware reranking improve the factual correctness and faithfulness of answers, relative to source-agnostic relevance-based retrieval over the same heterogeneous corpus?

  - **H2:** Validity-aware reranking will outperform source-agnostic retrieval by reducing the selection of outdated, weakly authoritative, or locally inapplicable evidence, with the largest gains on version-sensitive and current-policy queries.

A secondary analysis examines whether curated external documentation adds value for software-specific queries beyond the core local evidence base. To answer these questions, this project uses held-out comparisons of source ablations and reranking variants.

1.4 Aim, Objectives, and Success Criteria

The principal aim of this project is to design and evaluate an end-to-end RAG assistant for use in a supervised CSD3 support workflow. Within the scope of single-hop, text-accessible user support queries, the project investigates whether combining heterogeneous support evidence with validity-aware retrieval improves the factual correctness and faithfulness of grounded first-response answers. To achieve this aim, the project pursues the following objectives:

- **O1:** To construct a reproducible, queryable support corpus from local CSD3 documentation, historical support tickets, and selected external documentation, using source-specific preprocessing, chunking, and metadata enrichment.

- **O2:** To implement an end-to-end RAG assistant and baseline retrieval configurations for CSD3 support queries, capable of producing grounded first-response answers from retrieved evidence.

- **O3:** To design and integrate a validity-aware retrieval and reranking strategy that uses authority, scope, freshness, version, and status signals to prioritise evidence appropriate to the current CSD3 environment.
- **O4:** To develop a benchmark and experimental framework for single-hop CSD3 support queries, including a fixed development and test split and manually verified benchmark annotations for source requirements and validity sensitivity.
- **O5:** To evaluate whether heterogeneous-source retrieval and validity-aware reranking improve factual correctness over relevant baselines through controlled ablations, finalist comparisons, manual evaluation, and error analysis.

These objectives are evaluated against three research-facing success criteria. First, the project must provide a convincing held-out answer to whether combining local documentation and historical tickets improves factual correctness over single-source baselines. Second, it must provide a convincing held-out answer to whether validity-aware reranking improves over naive combined retrieval, judged primarily by factual correctness and checked against faithfulness. Third, it must reach a justified conclusion about the role of external documentation for software-specific queries, including whether any benefit is limited or conditional. Supporting implementation, reproducibility, and protocol criteria are operationalised in Chapter 3.

1.5 Overview of Evaluation Strategy

The evaluation strategy was designed to answer the research questions with minimal selection bias. The benchmark was split into development and held-out test partitions so that generator selection and configuration tuning were separated from final comparison [7]. The held-out experiments then compare single-source retrieval, naive combined retrieval, validity-aware reranking, and the contribution of external documentation. Factual correctness is the primary metric because the central aim of the project is to improve the correctness of support answers. Faithfulness is used as a guardrail against unsupported generation, while additional retrieval metrics are used diagnostically when comparing retrieval variants. Manual evaluation and error analysis complement automated scoring by identifying user-relevant failure modes and limitations [8–10].

The full experimental protocol is further described in Chapter 5, with results presented in Chapters 6 and 7.

1.6 Report Structure

The remainder of this report is structured to move from background and problem definition through to system design, evaluation, results, and conclusions. Chapter 2 reviews the literature, Chapter 3 defines the task, benchmark, and requirements, Chapter 4 describes the system, Chapter 5 presents the experimental methodology, Chapter 6 reports the results, Chapter 7 discusses the findings and limitations, and Chapter 8 concludes.

Chapter 2

Background and Related Work

This chapter introduces the relevant background and related work for this project, structured around its key design decisions. It starts by framing HPC support as an information retrieval and question answering problem, then introduces the concept of RAG and its relevance to this domain. The chapter also discusses the heterogeneous nature of support corpora, including documentation and tickets, and the challenges of authority, freshness, and validity in retrieval. Finally, it reviews existing evaluation methodologies for RAG systems, and positions this research within the broader landscape of related work.

2.1 HPC Support as an Information Retrieval and Question Answering Problem

Current user support for HPC systems such as CSD3 is typically delivered through a combination of documentation, helpdesk systems, and direct communication with support staff and HPC experts [2]. User documentation is a central component of HPC support and often serves as the first point of reference for users [2]. Bondapalli et al. [2] show that this guidance is often fragmented across multimodal sources, making relevant information difficult and time-consuming to locate. As HPC systems and software stacks continue to grow in complexity, even experienced users may need to integrate guidance from multiple sources, creating barriers to usability and increasing reliance on support channels [2]. This is particularly important in the context of HPC support, where incorrect guidance can result in misconfigurations, failed computations, wasted computational resources, or even violations of facility policies [2].

These characteristics make HPC support a natural Information Retrieval (IR) and QA problem [2, 11]. In the technical support setting, answering a user's query is a two-stage process: the system must first retrieve the relevant documents or support artefacts, and the retrieved information must then be read to identify the answer [11]. Specifically, the retrieved information must not only be semantically similar to the user's query, but it must also contain the required information to answer the question [11]. This is particularly important for procedural and troubleshooting questions, for which the answer is non-factoid and often requires actionable guidance rather than a memorised fact [11, 12]. The task is further complicated because the required documentation may be missing or difficult to locate in fragmented support environments [2, 11].

Technical support QA differs from general open-domain QA in several key ways [11]. First, it is grounded in real user queries that often refer to products, versions, configurations, commands, error messages, and stack traces [12]. Second, Yu et al. [11] argue that, in contrast to general open-domain QA, technical support QA often contains a weak lexical overlap between the question and the answer. Therefore, documents that appear semantically relevant may fail to contain the actual resolution. Straightforward retrieve-then-read pipelines may return such documents, making them more error-prone in technical settings [11]. Third, technical QA is constrained by relatively small labelled datasets, since annotation requires domain expertise and is significantly more expensive than crowd-sourced open-domain annotation [11, 12]. Taken together, these characteristics make direct transfer from methods trained on open-domain corpora more difficult and can lead to weaker performance in technical settings [11, 12].

In addition to local documentation, historical support tickets provide another source of operational support knowledge in HPC settings [13]. TicketHub shows that historical support tickets can help staff identify prior issues and resolutions, improving the efficiency of support responses [13]. The authors further show that improved access to archival tickets helps preserve operational knowledge and reduces the learning burden on newer staff, especially when institutional systems or workflows change. Therefore, HPC support depends not only on access to documentation, but also on access to prior support interactions and their resolutions [13].

Overall, the literature supports treating HPC support as a specialised form of technical QA in which answers depend on retrieving operationally usable evidence rather than recalling isolated facts [2, 11, 12].

2.2 Retrieval-Augmented Generation

As discussed in 2.1, technical support QA is evidence-dependent: answering a query requires retrieving the relevant external information and identifying an answer within it [11, 12]. For this reason, parametric-only Language Model (LM)s are a weak fit for knowledge-intensive support tasks, because they rely on knowledge stored in model parameters rather than directly verifiable external evidence [4]. As Lewis et al. [4] note, such models are difficult to update, provide limited provenance for their predictions, and can hallucinate on knowledge-intensive tasks. Retrieval-augmented generation (RAG) addresses this limitation by combining language generation with retrieval from an external knowledge source [4].

Lewis et al. [4] define RAG as a hybrid architecture that combines parametric memory in a pre-trained sequence-to-sequence generator model with non-parametric memory in a dense document index. At inference time, a neural retriever selects the most relevant passages for a given input query and the generator produces a response using these passages as additional context. RAG is therefore a retrieval-grounded generation framework intended to improve factual performance while making the evidential basis of an answer more inspectable [4].

Lewis et al. [4] report strong empirical results on open-domain QA, knowledge-intensive generation, and fact verification benchmarks. Their models outperform parametric-only baselines on several open-domain QA tasks and generate more factual, specific, and diverse responses than BART in generation settings [4]. Additionally, because knowledge is stored partly in an external document index, that knowledge can be updated by replacing or rebuilding the index rather than retraining the generator. Related work further shows that retrieval-augmented generation can reduce hallucination in grounded conversation settings [14].

However, the original RAG architecture does not by itself ensure that retrieval is necessary, that the retrieved material is trustworthy, or that the generator will faithfully use it in the final answer [5, 6, 15]. Asai et al. [15] argue that fixed-k retrieval can be wasteful or harmful when retrieval is unnecessary or when the retrieved passages are weakly grounded. Li et al. [5] show that RAG quality depends on query-knowledge relevance, and that it degrades when the underlying corpus is out of scope or out of date. Hwang et al. [6] further show that relevance alone is insufficient in multi-source settings, because source reliability affects whether retrieved evidence should be trusted. For technical support, this means that semantically relevant material may still be unusable if it is outdated or if it comes from a less reliable source [5, 6].

Approach	Control signal	What it improves	What remains unresolved for CSD3 support
Self-RAG [15]	Retrieval necessity and passage support	Avoids indiscriminate fixed-k retrieval; enables retrieval on demand	Does not model heterogeneous source roles, local authority, or temporal validity across support sources
Li et al. [5]	Query–knowledge relevance / corpus scope	Detects when queries fall outside the effective scope of the corpus or when the corpus is outdated	Does not rank competing sources by authority or local applicability once retrieval proceeds
Hwang et al. [6]	Source reliability	Prioritises reliable and relevant sources in multi-source RAG	Does not directly address support-specific temporal drift or the distinct evidential roles of documentation versus tickets

Table 2.1: The retrieval-control strategies most relevant to validity-aware support retrieval.

Taken together, these studies show that the key design problem is no longer whether to retrieve, but how retrieval should be controlled and which evidence should be prioritised [5, 6, 15]. These control strategies operate at different points in the pipeline: Self-RAG controls whether retrieval is necessary at all [15], Li et al. [5] assess whether the corpus is capable of supporting a query, and Hwang et al. [6] control which sources should be trusted once retrieval occurs. Together they suggest that reliable support RAG requires more than stronger semantic matching. It also requires explicit control over retrieval necessity, corpus scope, and source reliability. Table 2.1 summarises the retrieval-control strategies most relevant to this project, and highlights what each one leaves unresolved for heterogeneous CSD3 support.

The comparison shows that prior work addresses retrieval necessity, corpus scope, and source reliability as separate problems, but does not by itself resolve how these concerns should be combined for heterogeneous support evidence.

Applied support-oriented RAG systems also suggest that retrieval design remains domain-specific. AskHPC, for example, combines curated HPC documentation with specialised parsing [2], while customer-service systems have incorporated structured knowledge graphs to improve retrieval [16]. For HPC support, the implication is that a useful RAG system must not only retrieve semantically relevant information, but must also prioritise evidence that is current, trustworthy, and operationally valid [2, 5, 6].

Source type	Typical authority	Main strength	Main risk	Retrieval implication
Local documentation	Primary authority for site-specific practice	Records supported local workflows, constraints, and policy-relevant configuration [1, 2]	May omit low-level tool detail and edge-case troubleshooting [2]	Prefer for site-specific questions about access, authentication, job submission, permissions, and supported configuration.
External technical documentation	Secondary for local operational truth; strong on tool semantics	Provides tool behaviour, syntax, and configuration detail beyond local guides [2]	Can be relevant yet out-of-scope, out-of-date, or less reliable for local operational decisions [5, 6]	Use as supplementary evidence for software-specific questions; do not treat as decisive for local policy or permissions.
Historical support tickets	Variable: high for prior-case retrieval, weaker for current policy	Preserves concrete problem reports, diagnostic context, and prior resolutions [3, 13]	Lagging, noisy, and privacy-sensitive; may contain stale or user-specific content [3, 13]	Use with freshness controls and access restrictions; strongest for troubleshooting analogies and prior-case retrieval.

Table 2.2: Comparison of the evidential roles, risks, and retrieval implications of the three source types in the CSD3 support setting.

This motivates this project's focus on evidence prioritisation over heterogeneous support sources, rather than treating generation quality as the primary research problem.

2.3 Heterogeneous Support Corpora: Documentation and Tickets

HPC user support draws on a heterogeneous evidence base that includes local facility documentation, external technical documentation, and historical support tickets rather than a single uniform corpus [2, 3, 13]. In this context, heterogeneity means that these sources differ not only in topic coverage, but also in their evidential role, authority, update cycle, and temporal stability [2, 5, 6]. In multi-source retrieval settings, these differences affect whether apparently relevant evidence is still trustworthy, locally applicable, and suitable for the current system state [5, 6]. Source selection is therefore an important part of the retrieval problem [5, 6]. Table 2.2 presents the main evidential strengths, risks, and retrieval implications of the three source types considered in this project.

For facility-specific support, local documentation is the primary documentary source because it records the workflows, constraints, and configurations that are actually supported on the target system [1, 2]. AskHPC describes user documentation as the first point of reference for HPC users, which makes the quality, coverage, and maintenance of local documentation central to scalable support [2]. In the case of CSD3, the local documentation covers access, authentication, job submission, software environments, and site policies [1]. It is therefore the best available source for questions about permitted or supported local practice.

External technical documentation has a complementary evidential role because it can extend local support coverage without displacing local authority [2]. In multi-source RAG, however, such material still raises source-reliability and local-applicability questions [5, 6]. It is most useful when a query depends on tool behaviour, syntax, API usage, or configuration detail that local documentation omits, summarises only briefly, or abstracts away [2]. Bondapalli et al. [2] illustrate this multi-source setting clearly by combining facility guides with scheduler manuals and programming-model documentation to answer queries whose evidence requirements span multiple sources. However, general software documentation cannot by itself establish local operational truth, because current local practice may differ from the general case [2, 5, 6]. As a result, this project treats external documentation as supplementary evidence for CSD3-specific queries rather than as the deciding source on local policy, permissions, or configuration.

Historical support tickets perform a different evidential role to documentation, because they preserve concrete user problems, diagnostic context, troubleshooting paths, and prior resolutions in a way that formal documentation often does not [3, 13]. Rodenbeck and Suk Uhr [13] show that archival ticket analysis can help staff identify common issues, retrieve previous resolutions, and reduce the steep learning curve faced by new staff by making prior cases easier to search. Joslin et al. [3] further show that HPC tickets can be normalised into structured issue-resolution pairs and used to generate FAQ-style support knowledge from recurring support patterns. They also note that ticket corpora are difficult to prepare cleanly because they contain domain-specific terminology, cryptic error messages, inconsistent wording, and user-specific artefacts, all of which complicate normalisation, summarisation, and retrieval. Tickets are also a risky evidence source because they are a lagging indicator of system state. While a closed ticket may preserve a useful resolution, that resolution may no longer be valid after later changes to documentation, software versions, or local policy [5, 13]. Joslin

et al. [3] additionally highlight the privacy and governance constraints of ticket data, since personally identifying and user-specific content may persist even after redaction. Accordingly, ticket evidence often requires stronger access control, heavier preprocessing, and more careful freshness checks that documentation sources [3, 13].

The same passage may be relevant across documentation, external manuals, and historical tickets, but its evidential status changes with source role, local applicability, and temporal currency. For this project, this means indexing and ranking account for source type, local applicability, freshness, and evidential risk alongside semantic relevance [5, 6]. This motivates the validity-aware retrieval design developed in the next section.

2.4 Authority, Freshness, and Validity in Retrieval

Accordingly, the central retrieval question in this project is not simply whether a passage is semantically related to a query, but whether it should be treated as usable evidence for an answer [5, 6]. In a multi-source support corpus, relevance-only retrieval is inadequate because a highly relevant document may come from an unreliable source, or a seemingly appropriate corpus may still fail if the query lies beyond its effective scope or depends on outdated knowledge [5, 6]. This is especially problematic in operational support settings, where outdated information can reduce response accuracy and even steer generation towards harmful answers despite the simultaneous presence of current evidence [17]. Here, authority refers to the trustworthiness of a source, freshness to the temporal currency of its information, and validity to the suitability of retrieved evidence for the current local support context.

Authority becomes a retrieval variable once the corpus varies not only in topic coverage but also in evidential trustworthiness [6]. Hwang et al. [6] show that standard multi-source RAG can retrieve incorrect information because it relies on relevance alone while overlooking source reliability. This can allow unreliable sources to dominate the retrieval process. As discussed in Section 2.3, this is especially important for operational support questions about local policy, permissions, and supported configuration, where generic software documentation cannot by itself establish local operational truth. Authority is therefore a retrieval problem, not just a post-hoc verification problem: if authoritative evidence is not prioritised before generation, the model may still ground its answer in semantically relevant but less trustworthy material [6].

Freshness is important because the evidential value of a retrieved document can

decay over time, even when its semantic relevance remains high [5, 17]. Li et al. [5] show that RAG performance degrades when the underlying corpus is out of date or no longer adequately supports the active query distribution. Ouyang et al. [17] further demonstrate that the presence of outdated information in retrieved context can substantially reduce answer accuracy despite the simultaneous presence of current evidence. In HPC support, this temporal risk is amplified because the operational environment itself changes over time, as software stacks, documentation, policies, and even the available hardware evolve, meaning that older resolutions may no longer reflect the current system state [3, 13, 18]. Therefore, freshness is not only a useful ranking heuristic, but also a condition on whether retrieved evidence can still be safely used to answer present-focused operational queries [5, 17].

In this dissertation, validity is used as an operational term for retrieved evidence that is not only semantically relevant, but also authoritative, current, and locally applicable. This means that a passage can be topically relevant yet still be invalid if it comes from a less trustworthy source, reflects an outdated system state, or does not establish the correct local operational truth [5, 6]. This suggests that temporal validity should not be reduced to a simple age threshold alone, since the usability of support evidence depends on whether it still reflects the current local operational context. It also explains why indiscriminate fixed-k retrieval is risky in this setting: adding more semantically related context does not guarantee better grounding, and the generator is not guaranteed to follow the retrieved evidence faithfully [15]. Validity-aware retrieval should therefore prefer evidence that is jointly relevant, reliable, current, and applicable to the local support context, rather than treating semantic similarity alone as sufficient ranking criteria [5, 6, 15].

Taken together, these studies imply that retrieval for HPC support should not be organised as a relevance-only search over a flattened corpus. Instead, it should incorporate source-aware preferences, freshness-sensitive controls, and local applicability constraints so that generation is grounded primarily in evidence that is defensible in the current support context [5, 6, 17]. They also suggest that when retrieved evidence is weak, stale, or conflicting, conservative response behaviour is preferable to forced resolution [5, 15]. Therefore, this project treats validity-aware retrieval as a prerequisite for trustworthy support generation, which motivates the retrieval and reranking design developed in Chapter 4.

2.5 Evaluation of RAG Systems

Evaluating RAG systems is more complex than evaluating standalone LMs because quality depends both on what evidence is retrieved and on how faithfully the generator uses that evidence [8, 19]. A single end-to-end score is therefore insufficient because a poor final answer could be caused by retrieval failure, incomplete evidence, or weak use of otherwise valid context [19]. As a result, recent work treats RAG evaluation as a multi-dimensional problem that separates retrieval quality from grounded generation quality [8, 20]. This is typically operationalised through measures such as context relevance, answer faithfulness, and answer relevance [8].

RAGAS provides a scalable, reference-free framework for evaluating RAG systems using three core dimensions: faithfulness, answer relevance, and context relevance [8]. This decomposition is especially useful during system development, because it enables rapid comparison of RAG systems even when human-annotated references are unavailable [8]. ARES challenges the assumption that prompt-based judge frameworks are automatically reliable, showing that evaluation improves when judges are tailored to the target setting and calibrated with a small human-annotated validation set [20]. Automated metrics are therefore valuable for scalable iteration, but they still require scrutiny over judge design and validation before they should be trusted for strong comparative claims [8, 20].

Even when automated frameworks report several quality dimensions, evaluation still remains too coarse if it only produces overall scores without showing where failure arose [19]. Ru et al. [19] therefore argue that RAG evaluation should serve both user-facing comparison and developer-facing diagnosis, because errors may originate in retrieval, generation, or in the interaction between the two. RAGChecker extends this line of work by moving from response-level assessment to claim-level entailment checking and by separating overall metrics from diagnostic retriever and generator metrics [19]. This is particularly important for system development, since fine-grained evaluation can expose trade-offs that aggregate scores hide, such as retrieval improvements that also introduce noise or generators that appear strong overall but still produce unsupported claims [19].

Metric choice alone does not make RAG evaluation convincing. In particular, the benchmark must also expose the retrieval conditions under which the systems fail [21]. Ben [21] address this directly in their Retrieval-Augmented Generation Benchmark (RGB) benchmark by organising evaluation around four fundamental abilities for RAG: noise robustness, negative rejection, information integration, and counterfactual

robustness. This choice of evaluation cases is especially valuable since it tests not only whether a model can answer when helpful evidence is present, but also whether it can refuse when evidence is missing, combine evidence across documents, and resist misleading retrieved content [21].

Taken together, the literature supports that strong RAG evaluation should combine scalable automated scoring with smaller amounts of human-grounded validation and targeted diagnostic analysis, rather than relying on any single framework or headline metric alone [8, 19, 20]. Automated frameworks remain valuable because they make the iterative comparison of RAG configurations practical at development time, especially when large-scale human annotation is unavailable [8, 20]. However, both the limitations of Large Language Model (LLM)-as-a-judge methods and the importance of diagnostic analysis indicate that automated scores should be interpreted with caution and complemented by human judgement and failure analysis when drawing stronger comparative conclusions [8, 19, 20]. These studies justify the evaluation design developed in Chapter 5, which uses automated metrics for scalable ablation and comparison, while retaining manual evaluation and error analysis to assess response quality and identify failure modes.

2.6 Gap Analysis and Research Positioning

The reviewed literature positions HPC support as a specialised retrieval-grounded QA problem rather than a generic chatbot task [2, 11]. AskHPC show that HPC support information is scattered across multiple sources [2], while Yu et al. [11] frame technical support QA as a retrieve-then-read process in which relevant evidence must be found before an answer can be identified. This is important because answer quality depends on appropriate external evidence, while technical-support datasets are often limited by the cost of expert annotation [11, 12].

This chapter has also shown that the support evidence base is heterogeneous rather than uniform, with each source contributing a different type of information [2, 3, 13]. In particular, local documentation, external technical documentation, and historical support tickets provide current site-specific guidance, supplementary software detail, and prior issue-resolution experience respectively [2, 3, 13]. Furthermore, since these sources differ in authority, temporal stability, and local applicability, they cannot be treated as interchangeable [3, 5, 6].

The heterogeneity of the underlying support corpus exposes a limitation in baseline

RAG architectures. Although baseline RAG improves knowledge-intensive generation by grounding outputs in external knowledge [4], later work shows that reliable RAG also depends on whether retrieved evidence is in scope, current, and trustworthy [5, 6, 17]. In particular, Li et al. [5] show that degradation occurs when the underlying corpus is out of scope or out of date, Hwang et al. [6] show that relevance-only retrieval overlooks source reliability, and [17] show that outdated evidence can reduce accuracy and mislead generation even when valid evidence is present.

The resulting literature gap is therefore not that HPC support has never been addressed using retrieval-augmented generation. AskHPC, for example, demonstrates the value of documentation-grounded RAG for HPC user support, and ticket-focused studies highlight the value of historical support interactions [2, 3, 13]. This review did not identify prior work that jointly evaluates retrieval-time evidence prioritisation across documentation and tickets when those sources differ in freshness, authority, and local validity [5, 6, 17]. More specifically, this review did not identify prior work that tests whether answer quality improves when heterogeneous support sources are combined with validity-aware reranking rather than source-agnostic semantic retrieval.

The dissertation is therefore positioned against a narrow gap defined by the intersection of heterogeneous support evidence, validity-aware prioritisation, and supervised first-response assistance for CSD3. Within this scope, the project tests whether combining documentation and tickets improves factual correctness over single-source baselines, and whether validity-aware reranking improves over naive combined retrieval.

The literature further implies that evaluation must mirror the structure of the problem. Es et al. [8] argue that RAG quality cannot be reduced to a single end-to-end score, while Ru et al. [19] show that useful diagnosis requires separating retriever and generator failure modes. As a result, the later chapters evaluate retrieval quality, faithfulness, factual correctness, and error patterns rather than relying on one headline metric.

Chapter 3

Problem Formulation and Requirements

3.1 Task Definition

This section formulates CSD3 user support as a single-hop, evidence-grounded question-answering task within a supervised internal support workflow. Given a natural-language support query and a heterogeneous evidence base consisting of local documentation, selected external documentation, and historical support tickets, the system must retrieve evidence passages that are not only semantically relevant, but also sufficiently authoritative, current, and locally applicable to the present CSD3 environment. It must then generate a first-response answer suitable for staff-assisted use: factually correct, faithful to the retrieved evidence, and appropriately qualified for the local operational context. This framing is consistent with retrieval-augmented generation for knowledge-intensive tasks, where answer quality depends on access to external evidence rather than parametric memory alone [4], and with technical support QA, which is naturally structured as a retrieve-then-read problem [11]. It also reflects recent findings that apparently relevant evidence may still be unsuitable if it is out of scope, out of date, or drawn from less reliable sources [5, 6].

In this dissertation, a query is treated as single-hop when it can be answered from one passage, or a small set of closely related passages, retrieved in a single retrieval step, without subsequent query reformulation, multi-stage decomposition, or chained intermediate retrieval. The task is further scoped to documentation text and visible ticket text. Uploaded attachments and embedded media, including screenshots and log files, are excluded from retrieval even where they contain text. This is a methodological

restriction that was chosen to reduce confounding from evidence-composition failures and multimodal interpretation errors, so that performance differences can be attributed more cleanly to heterogeneous-source retrieval and validity-aware reranking. It also keeps the benchmark reproducible by ensuring that all evaluated conditions operate over the same visible textual evidence.

The intended output of the system is not an autonomous final resolution, but evidence-grounded first-response assistance for realistic CSD3 support queries. The system is therefore designed for use within a supervised workflow, where retrieved evidence and generated answers can be reviewed by support staff before action is taken. Within this setting, the system should prefer cautious, qualified assistance when the available evidence is incomplete, conflicting, stale, or otherwise uncertain, rather than forcing an overconfident resolution. This operational framing aligns the task with the practical requirements of internal HPC support while preserving a controlled scope for later evaluation.

3.2 Data Sources and Knowledge Characteristics

The evidence base for this project consists of three heterogeneous source classes: local documentation, selected external documentation, and historical support tickets. These sources were indexed as source-distinct corpora rather than flattened into a single undifferentiated text pool, because they differ in evidential role, authority, temporal stability, and local applicability. Table 3.1 summarises the operational role and retrieval implications of each source class.

Local CSD3 documentation formed the primary documentary source in the system, since it records the workflows, policies, access conditions, and locally supported configurations that define current site-specific practice [1, 2]. Operationally, the local corpus was restricted to internal HTML pages reachable from the CSD3 documentation homepage within the `docs.hpc.cam.ac.uk/hpc/` subtree. This restriction prioritised pages that directly express current local operational guidance and excluded loosely related material outside the core documentation boundary. The resulting frozen local documentation snapshot used in parameter tuning and final evaluation contained 44 pages. In retrieval terms, this source acts as the default authority for questions about what is currently supported on CSD3.

Selected external documentation formed a supplementary documentary source, extending coverage when queries depended on software-specific detail not fully captured

Source class	Evidential role	Inclusion basis and scale	Main exclusions	Retrieval implication
Local documentation	Primary record of current site-specific practice	Internal HTML pages within the core CSD3 documentation subtree; 44 pages	Out-of-subtree and non-HTML material	Highest local authority; should dominate questions about supported workflows, policy, access, and configuration
Selected external documentation	Supplementary software and toolchain detail	Stable official manuals relevant to the CSD3 support domain; 13 sources, 662 pages	Community-managed, non-official, overly broad, or misaligned subtrees	Useful for software behaviour, syntax, and APIs, but should not override local policy or configuration evidence
Historical support tickets	Prior-case troubleshooting evidence from resolved interactions	Resolved CSD3 support tickets from November 2017 to 1 April 2026 inclusive; 89,905 tickets	Unresolved tickets and attachment/media contents as retrievable evidence	High coverage for recurring issues and diagnostic context, but noisier, more privacy-sensitive, and more temporally fragile than documentation

Table 3.1: Operational summary of the three source classes included in the project corpus. Counts and date bounds refer to the frozen corpus snapshot used for parameter tuning and final evaluation. The full external source list is given in Appendix A.

in the local documentation [2]. Sources were included only when they were stable official manuals for software or toolchains relevant to the CSD3 support domain and likely to contribute detail on software behaviour, syntax, APIs, or configuration beyond the local corpus. Community-managed, non-official, overly broad, or otherwise misaligned documentation subtrees were excluded. The approved set was defined in an external source registry and the frozen external documentation snapshot used for parameter tuning and final evaluation contained 13 sources, with a total of 662 pages. The full approved source list is provided in Appendix A. While these documents expand technical coverage, they do not displace local documentation as the deciding source on site-specific policy, permissions, or configuration.

Historical support tickets formed the third source class, providing prior-case evidence from past support interactions that may contain diagnostic context, troubleshooting paths, and previously successful resolutions [3, 13]. This project considered all resolved CSD3 support tickets from November 2017 to 1 April 2026 inclusive. Only tickets with the Jira status `Resolved` were ingested. Attachment and embedded media content were not semantically interpreted during ingestion. Instead, media references were converted into textual placeholders indicating that an attachment was present without treating its contents as retrievable evidence. This preserved ticket conversation context while keeping the corpus within the text-accessible scope defined in Section 3.1. The resulting frozen ticket snapshot used for parameter tuning and final evaluation contained 89,905 tickets. Compared with documentation, tickets provide richer prior-case detail, but are also noisier, more lexically variable, more privacy-sensitive, and more vulnerable to temporal decay [3, 13].

Taken together, these source classes were deliberately retained as distinct evidential categories rather than being merged into a single homogeneous corpus. Source identity, temporal fields, and authority-related metadata therefore needed to be preserved so that later retrieval and reranking could prefer current local documentation for site-specific questions, use external manuals as supplementary software evidence, and exploit historical tickets primarily as prior-case troubleshooting support.

3.3 Benchmark and Query Taxonomy

This section describes the construction of the benchmark dataset used for parameter tuning and final evaluation, along with the query taxonomy used for later performance analysis. This benchmark was designed to reflect realistic CSD3 support queries while remaining within the single-hop, text-accessible scope defined in Section 3.1.

To ground the benchmark in realistic operational demand, the Head of HPC Services at the University of Cambridge Research Computing Services was consulted to identify a comprehensive set of realistic user-support topics. This consultation resulted in a set of 47 representative support-topic keywords spanning a broad range of common CSD3 support themes. These keywords were used as seed topics for benchmark construction, rather than for the final query taxonomy. The 47 keywords were queried through the Jira API to retrieve candidate tickets, ordered by resolution date with the most recent first. For each keyword, candidate tickets were screened manually, with duplicates across keyword searches removed before final selection. Candidate tickets

were then manually reviewed for inclusion based on realism, relevance to CSD3 support, diversity of query types, and alignment with the scoped task definition. Recently resolved tickets were prioritised to ensure that the benchmark reflected current support demand and could be answered from the frozen evidence snapshot. Tickets were excluded if they were duplicative, clearly out of scope, dependent on unseen attachments, or required additional information that would not be available to the system at inference time. The final benchmark size of 100 queries was chosen to preserve broad topical coverage while keeping later evaluation tractable.

The resulting benchmark dataset contained 100 standalone evaluation queries, each derived from the initial user-facing query of a selected support ticket. This prevented trivial retrieval of the original ticket and reduced benchmark contamination, ensuring that system performance reflected generalisation over the remaining evidence base rather than recovery of a near-duplicate source instance. This benchmark therefore follows the general logic of test-collection-based IR evaluation, in which a fixed query set is evaluated against a fixed evidence corpus [22]. However, it is not a full Cranfield-style collection, since no exhaustive relevance judgements or fixed gold contexts were identified for each query. No fixed reference passage set was attached to each query, since in the heterogeneous support setting there may be multiple valid evidence combinations that could support a correct answer. Pre-specifying a single target context would therefore impose an artificially narrow notion of retrieval success.

To support later subgroup analysis, the benchmark taxonomy was implemented as a multi-label annotation scheme rather than as a single topic label. Each query was annotated across seven categories: the minimum evidence source needed for a correct answer; the required documentary scope; whether correctness depends on current local operational truth; whether unseen attachments materially affected the response; the broad query family; whether the query was explicitly version-sensitive; and whether the answer depended on a particular system, service, partition, or platform. The full label definitions and allowed values are provided in Appendix B. All taxonomy labels were assigned manually by the author. However, this benchmarks remains subject to several limitations: topic labels were curated from a single expert rather than through a consensus process; keyword-based candidate retrieval may miss differently phrased tickets, and manual curation and annotation introduce researcher bias.

3.4 Requirements and Constraints

This section formalises the practical requirements and constraints that follow from the task definition, heterogeneous evidence setting, and benchmark methodology developed in Sections 3.1 to 3.3. It translates the problem formulation into explicit system requirements against which later design and evaluation decisions can be judged. To support a more rigorous methodology, the requirements are grouped into three classes: functional requirements, which define what the system must do; quality requirements, which define the standard the system should achieve; and constraints, which define the intended scope and limits of the project. These requirements and constraints are summarised in Table 3.2, which also provides the rationale needed for direct mapping into the success criteria defined in Section 3.5.

Table 3.2: Functional requirements, quality requirements, and scope constraints derived from the project task definition and benchmark design.

ID	Requirement	Type	Rationale
R1	The system must retrieve candidate evidence from all three source classes: local documentation, selected external documentation, and historical support tickets.	Functional	The task is defined over a heterogeneous evidence base, so retrieval must not assume a single uniform corpus.
R2	The system must rank retrieved evidence using source-aware validity criteria rather than semantic relevance alone.	Functional	In this setting, relevant evidence may still be outdated, non-authoritative, or locally inapplicable.
R3	The system must generate an evidence-grounded first-response answer rather than an unsupported free-form response.	Functional	The project concerns staff-assisted support, not general conversational helpfulness.
R4	The system must support supervised use by producing outputs that can be reviewed by support staff.	Functional	The intended deployment setting is assistive rather than autonomous.
Q1	Answers should be factually faithful to the retrieved evidence.	Quality	Hallucinated or weakly grounded answers are especially harmful in technical support.
Q2	The system should prioritise locally authoritative and current evidence when these conflict with more generic or outdated sources.	Quality	Local operational truth has priority over generic or outdated documentation.

Continued on next page

Table 3.2 continued from previous page

ID	Requirement	Type	Rationale
Q3	The system should avoid overconfident resolution when the available evidence is incomplete, conflicting, or uncertain.	Quality	Overconfident but incorrect support guidance is operationally risky.
Q4	The benchmarked system configuration should be reproducible from frozen corpora and benchmark definitions.	Quality	Evaluation claims are only defensible if the setup is stable and auditable.
C1	The task is restricted to single-hop support queries.	Constraint	This keeps the problem controlled and avoids confounding from multi-stage reasoning.
C2	The task is restricted to evidence available in document text and visible ticket text.	Constraint	This keeps all evaluated conditions within the same visible evidence space.
C3	Attachments and embedded media are excluded from semantic interpretation.	Constraint	This avoids introducing multimodal processing as an uncontrolled source of variance.
C4	The evaluation does not assume exhaustive gold retrieval contexts for benchmark queries.	Constraint	In this heterogeneous setting, multiple valid evidence paths may support a correct answer, making a single exhaustive target context inappropriate.
C5	The system is not required to autonomously resolve out-of-scope, multi-stage, or attachment-dependent cases.	Constraint	These cases fall outside the project's controlled task definition.

These requirements define the formal contract for the project rather than a broad statement of aspiration. They specify both the intended capabilities of the system and the conditions under which its evaluation claims can be considered valid. Section 3.5 will later map each requirement into corresponding success criteria and evaluation conditions.

3.5 Success Criteria and Requirement-to-Evaluation Mapping

This section translates the functional requirements, quality requirements, and constraints defined in Section 3.4 into explicit success criteria and defines how each of these criteria will be evaluated. It formalises the success criteria already stated in

Chapter 1 by mapping `SC1` to `SC7` onto the requirements of the project. Table 3.3 summarises this mapping, together with the evaluation method and decision bases used to judge whether each criterion has been met. All comparative claims are made under a frozen development and test protocol in which development data is used for tuning and design decisions, while final conclusions are based on held-out test results.

Table 3.3: Success-criteria and requirement-to-evaluation mapping for the project.

SC	Req. IDs	Success criterion	Evaluation method	Decision basis
SC1	R1, R3, Q1	The project provides a convincing held-out test-set answer to whether combining local documentation and historical support tickets improves factual correctness over single-source baselines.	Comparative benchmark runs under a frozen development/test protocol, with development data used for tuning and held-out test data used for final comparison.	Met if held-out test results support a clear comparative conclusion about combined retrieval versus single-source baselines, judged primarily by answer correctness and checked against faithfulness.
SC2	R2, Q1, Q2	The project provides a convincing held-out test-set answer to whether validity-aware reranking improves over naive combined retrieval.	Comparative ablation runs on the held-out benchmark, including subgroup analysis of validity-sensitive queries.	Met if held-out results support a justified conclusion about the effect of validity-aware reranking, whether the observed effect is positive, neutral, or conditional.
SC3	R1, Q2	The project reaches a justified conclusion about the role of external documentation for software-specific queries.	Source-ablation subgroup analysis on benchmark subsets requiring external or mixed documentary scope.	Met if the evaluation shows, on held-out queries, whether external documentation is beneficial, unnecessary, or only conditionally useful for the scoped query class.
SC4	R3, R4, C1, C2, C3, C5	A working end-to-end RAG assistant is implemented for supervised, single-hop, text-accessible CSD3 support QA.	End-to-end system demonstration and benchmark execution on in-scope queries.	Met if the system can execute the full pipeline, produce reviewable first-response answers, and remain within the scoped task definition.

Continued on next page

Table 3.3 continued from previous page

SC	Req. IDs	Success criterion	Evaluation method	Decision basis
SC5	R1, Q4	The system can ingest and retrieve from heterogeneous support sources in a reproducible and source-distinct way.	Corpus audit, metadata inspection, and source-specific retrieval runs under common system settings.	Met if local documentation, external documentation, and historical tickets can each be indexed, retrieved, and analysed separately and in combination, with source identity preserved.
SC6	Q4, C1, C2, C3, C4	A fixed benchmark, verified annotations, and a frozen development/test evaluation protocol are produced for the in-scope query class.	Audit of benchmark files, split definitions, annotation codebook, and review-status controls.	Met if final experiments use the frozen benchmark artefacts and only manually verified annotations, with no post hoc change to test queries or evaluation protocol.
SC7	Q1, Q3, Q4, C4, C5	The final conclusions are supported by controlled experiments, reproducible artefacts, and honest failure analysis.	Review of experimental controls, frozen artefacts, held-out test results, and structured error analysis.	Met if the final claims are tied directly to the held-out evaluation, reproducible inputs, and explicit analysis of limitations, uncertainty, and failure cases.

Not all success criteria in Table 3.3 are evaluated in the same way. Some are primarily comparative and quantitative, particularly those concerning source combination, validity-aware reranking, and the role of external documentation. Others are primarily compliance or quality criteria, assessed through corpus audits, benchmark controls, faithfulness judgments, and structured error analysis. Therefore, success does not require perfect performance on every benchmark query. Instead, it requires that the held-out evaluation provides a justified answer to the project's research questions, that the system behaves credibly within the scoped task definition, and that the final conclusions are supported by reproducible artefacts and honest analysis of limitations and failure cases.

3.6 Ethical, Privacy, and Data-Governance Considerations

This project uses real internal support tickets as a primary data source. As a result, its main ethical concerns are privacy, confidentiality, and the risk of unsafe operational

advice. Historical tickets were not anonymised before ingestion, however this is justified by the intended development setting: the system is designed only for internal use within the CSD3 support service, where authorised staff already handle these records as part of normal support operations. Preserving the original ticket text also avoids distorting the operational context needed for realistic retrieval and evaluation. However, although the internal ticket corpus was not anonymised for system development, any ticket excerpts or illustrative examples included in the dissertation will be anonymised and minimised to avoid disclosing unecessary personal or operationally sensitive information. Risk was further reduced through governance and scope control: access was limited to internal project use, frozen corpora and benchmark artefacts were not intended for public release, attachment contents were excluded from semantic interpretation, and the system was framed as supervised first-response assistance rather than autonomous resolution. Residual risks remain. In particular, retrieved ticket text may contain personal information or incomplete context, and that generated answers may still be wrong. Furthermore, personally identifiable information present in retrieved ticket text could be reproduced unnecessarily in generated answers. These risks are acknowledged explicitly and mitigated through restricted access, staff oversight, and evaluation focused on faithfulness and cautious behaviour.

Chapter 4

System Design and Implementation

4.1 System Overview

This chapter describes Polaris, a retrieval-augmented assistant designed for supervised CSD3 support question answering. It implements the design requirements defined in Chapter 3 as a modular retrieval-augmented support system. Its architecture separates offline evidence preparation from runtime query handling so that corpus construction, metadata enrichment, retrieval, generation, and auditability can be controlled independently. The system indexes local documentation, external official documentation, and historical support tickets into source distinct stores, with authority and validity metadata used to preserve differences in reliability and operational relevance. This separation is central to Polaris's design, allowing runtime retrieval to combine heterogeneous evidence while still exposing the metadata needed for reviewable support responses.

Figure 4.1 separates Polaris into offline corpus construction and runtime answer generation. Offline construction ingested the evidence sources, converted them into a consistent representation, enriched them with metadata, and indexed them into source-distinct Qdrant collections together with per-source document stores [23]. At runtime, the FastAPI service coordinated registry-backed query interpretation, multi-collection retrieval, reranking, context resolution, prompt construction, LLM generation, and response packing. The observability layer recorded traces, prompt versions, feedback, and run artefacts, supporting reproducibility and making system behaviour auditable during evaluation and debugging.

Figure 4.2 illustrates how an individual user query moves through the system. A query is first validated by the API and associated with runtime tracing information. The system then extracts registry-backed constraints, such as software, version, scope, and

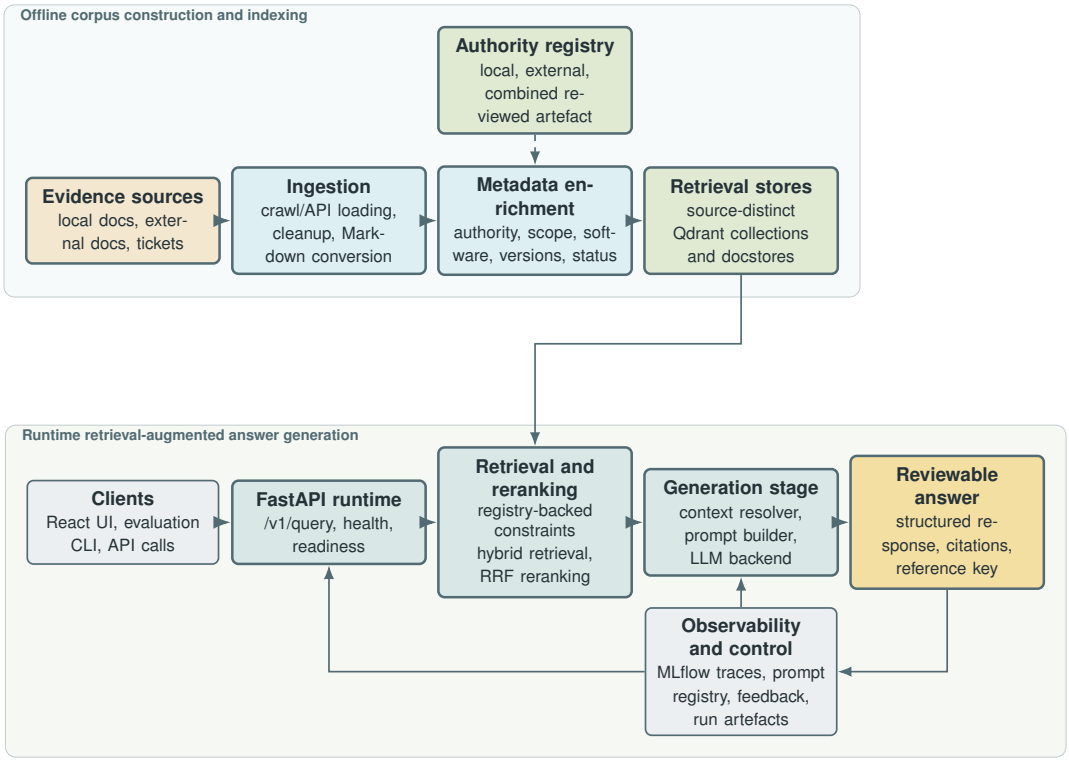

Figure 4.1: High-level architecture of Polaris. Offline corpus construction converts local documentation, external official documentation, and resolved support tickets into source-distinct retrieval stores enriched with authority and validity metadata. At runtime, the API coordinates registry-backed query-constraint extraction, multi-collection retrieval, configurable reranking, context resolution, prompt construction, generation, and reviewable evidence-grounded response output.

intent signals, before retrieving evidence from the configured source-distinct stores. Retrieved candidates are then fused and ranked using Reciprocal Rank Fusion (RRF), with validity signals available where enabled. The selected evidence is resolved into final contexts, rendered into the generation prompt, and passed to the LLM. The generated answer is then packaged by trimming output drift, deriving an evidence status from the retrieved context count, and logging the relevant outputs for auditability.

The remainder of this chapter decomposes this overview into the main implementation decisions and design trade-offs made during Polaris's development.

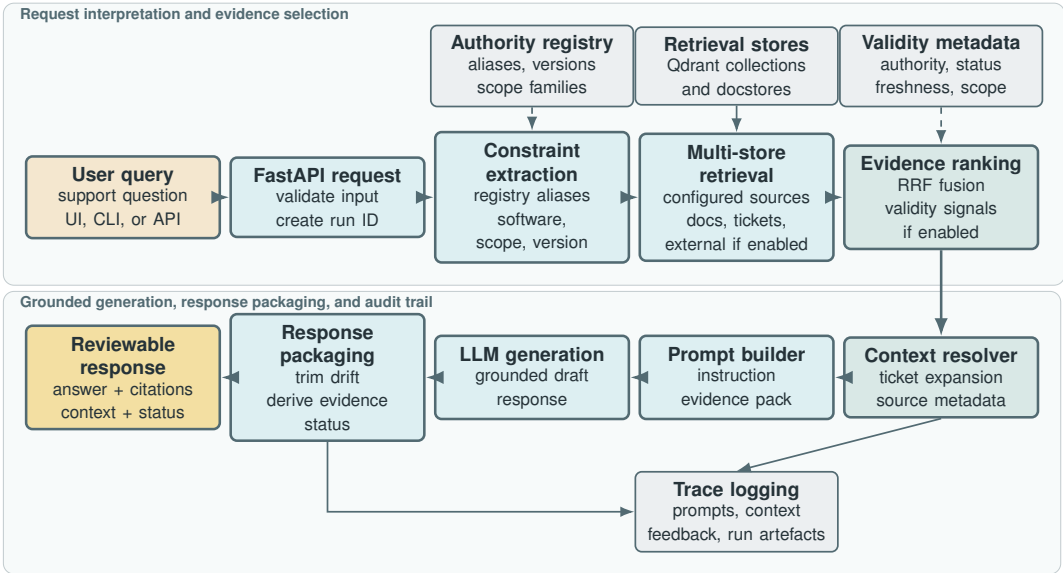

Figure 4.2: Runtime query flow in Polaris. A user query is validated by the API, converted into registry-backed retrieval constraints, matched against configured source-distinct stores, fused and reranked, resolved into final evidence contexts, passed to the generation prompt, and repackaged with output cleanup, evidence status, and trace outputs before a reviewable response is returned.

4.2 Baseline Systems

This project implements a series of baseline systems to evaluate the impact of specific design choices in Polaris. Each baseline system is a controlled comparison version of the main Polaris system, with specific design choices removed to isolate their effects. The generator, API pipeline, prompt format, and evaluation protocol are held constant across all systems to ensure that any observed differences in performance can be attributed to the design choices being evaluated.

Two baseline systems were implemented for Research Question 1. The first, `docs_only`, retrieved only from the local CSD3 documentation collection and measured how far local documentation alone could support the benchmark queries. The second, `tickets_only`, retrieved only from the historical support ticket collection and measured how far historical resolutions alone could support the same query set. Comparing these baselines with the combined system allowed the evaluation to test whether heterogeneous retrieval improved answer quality for the benchmark query mix, or whether one source dominated once the generator, prompt, and API pipeline were held constant.

A single baseline system was implemented for Research Question 2. This baseline,

`naive_combined`, used both local documentation and historical tickets with standard RRF retrieval but without metadata-driven reranking. It therefore acted as the direct implementation comparator for the proposed validity-aware reranking design. Additional retrieval variants were also implemented as ablations, but were not treated as formal baselines.

4.3 Corpus Ingestion and Representation

This section describes how raw evidence sources were ingested into retrieval-ready evidence units. Local CSD3 documentation, external documentation, and resolved support tickets were ingested through separate pipelines, but source boundaries were preserved so that later source-aware retrieval and reranking remained explicit.

In this dissertation, chunking referred to splitting a longer source document into smaller bounded text units that could be independently embedded, indexed, and retrieved. This choice mattered because whole-document retrieval was too coarse, while overly small chunks risked losing important context [24].

For the documentation sources, the ingestion pipeline first preprocessed the HTML pages and then converted them into a markdown-normalised representation before token-based chunking. Markdown conversion was used because the source material originated from heterogeneous HTML structures, whereas markdown preserved useful structure such as headings, lists, code blocks, and links in a consistent text format while removing much of the presentation-specific noise of raw HTML. The final documentation setting used 400-token chunks with no overlap. The same setting was used for the external documentation source.

For historical support tickets, the ingestion pipeline first queried the Atlassian Jira API for resolved tickets, converted the JSON content into a structured transcript format, and then applied a dialogue-based chunking strategy. A dialogue-based representation was used because important support evidence often spans short sequences of messages rather than a single isolated passage, and prior work has shown that historical tickets preserve useful issue-resolution context for support workflows [3, 13]. Each ticket was represented using its key, summary, status, available timestamps, initial description, and subsequent conversational exchange. Speaker roles were derived from Jira user fields and preserved at the message level. Ticket-level status and timing information were retained as metadata rather than embedded directly into every chunk. In the final system, ticket chunks used a 1000-token budget, 80 tokens of overlap, and

a maximum of three distinct turns per chunk. Retrieved ticket chunks were then used to identify relevant tickets, after which the corresponding full ticket transcript was resolved for generation so that retrieval remained fine-grained while answer generation included the broader problem-resolution context.

The final chunking settings were selected empirically on the development split. Documentation-only tuning selected 400 tokens with zero overlap because it gave the highest mean factual correctness, 0.299, among the tested documentation settings. Ticket-only tuning selected 1000 tokens with 80 tokens of overlap because it gave the highest mean factual correctness, 0.326. A combined follow-up then retained the 400/0 plus 1000/80 pairing, which produced the highest mean factual correctness, 0.329, of the tested mixed settings. Section 5.6.2 reports the full comparison.

4.4 Authority Registry and Metadata Extraction

Once the source documents have been converted into retrieval-ready representations, the system requires a structured authority layer to identify the technical entities they mention and to attach the metadata needed for later retrieval decisions. This is implemented as an offline authority registry that normalises systems, partitions, services, software packages, modules, and toolchains together with their associated scope, version, and lifecycle information. During ingestion, this registry is used to enrich source records before their metadata is propagated to the retrieval chunks.

To ensure deterministic and auditable metadata enrichment, the authority registry is built offline, prior to runtime query handling. Local CSD3 documentation, and where configured, registered external documentation sources are first converted into separate markdown-normalised corpora. These corpora are scanned for candidate entities, which are then merged into normalised registry entries so that repeated mentions across pages map to a single stable definition. Ambiguous aliases or conflicting lifecycle statuses are not resolved silently. Instead, they are added to a review queue for manual inspection. The resulting artefacts are persisted as stable JSON along with build metadata such as source scope and extraction version, supporting traceability and reproducibility. Table 4.1 summarises the entity categories captured by the registry and the main classes of information stored for each entry.

Table 4.1: Summary of the entity categories and metadata stored in the Polaris authority registry.

Registry aspect	Contents	Why it is stored
Entity categories	systems, partitions, services, software, modules, toolchains	Normalises the main technical entity types that Polaris must reason about across documentation and tickets.
Identity fields	canonical name, aliases	Links different surface forms to the same underlying entity during matching and query interpretation.
Scope, lifecycle, and version fields	source scope, lifecycle status, known versions	Supports source-aware retrieval and version-sensitive reasoning.
Provenance fields	source document, heading path, evidence span	Makes extracted metadata traceable to the originating source and section.
Audit fields	extraction method, review state	Supports reproducibility and manual review of ambiguous or conflicting cases.

For the documentation sources, metadata extraction is mainly done by matching the document's source identity against registry entries so that each chunk inherits the entity information associated with its originating document. On the other hand, for the ticket sources, metadata extraction is text-driven: ticket content is scanned using alias matching, `module load` parsing, and version mention extraction. The resulting matches are then resolved against the same authority registry. This process attaches authority, entity, scope, validity, freshness, provenance, and governance signals to each retrieval unit, with the complete field-level schema summarised in Appendix C. Using the same registry for both documentation and ticket metadata extraction ensures that they are both interpreted through one consistent set of entity definitions, rather than source-specific rules.

This metadata is not stored solely for description, but also to support later runtime decisions. Since each retrieval unit carries explicit authority, scope, version, validity, and provenance signals, the same registry-backed representation can be used for query constraint parsing, source-aware retrieval, and validity-aware reranking. The authority layer therefore acts as a bridge between offline corpus preparation and online answer

generation. Furthermore, the authority layer also improves auditability, since retrieved evidence can be traced back to its original source, matched entities, and extraction context, making errors easier to inspect during evaluation and debugging.

4.5 Validity-Aware Reranking Design

As discussed in Chapter 2, semantically relevant evidence could still be unsuitable if it was less authoritative, outdated, or locally inapplicable [5, 6]. The system therefore did not treat retrieval as a purely relevance-based problem over a flattened homogeneous corpus. Instead, initial semantic retrieval was followed by a metadata-driven reranking stage that could promote evidence that was more authoritative, current, and locally applicable. This design was implemented as a testable hypothesis rather than being assumed to improve the results prior to evaluation.

After query constraints had been derived, the system retrieved candidates using a shared hybrid budget of eight dense retrievals and eight BM25 retrievals per source. Per-source rankings were fused using Reciprocal Rank Fusion (RRF) with $k = 60$, preserving source identify for later tracing, and the fused candidate set was truncated to a final top 8 before context resolution [25]. The semantic relevance-only fused ranking corresponded to the `naive_combined` baseline introduced in Section 4.2 and served as the direct implementation comparator for the reranking design.

As illustrated in Figure 4.3, the validity-aware reranker began with the normalised semantic base score derived from the RRF ordering. The final reranking score was formed by adding weighted metadata features to this semantic base. In the final experimentally tuned configuration, authority and exact scope match each received weight 0.08, lifecycle status and version each received 0.04, scope-family received 0.02, and relative freshness received 0.01. Software match was implemented by received zero weight in the selected configuration. Authority values were mapped as 1.0 for local CSD3 documentation, 0.5 for external documentation, and 0.0 for historical tickets. Lifecycle values were mapped as 1.0 for current, 0.25 for maintenance, -0.5 for legacy, and -1.0 for end-of-life.

These validity signals were not applied blindly to every query. The reranker used the derived query constraints to control how individual metadata features behaved. This was especially important for software-constrained queries, where a broad scope-family match could otherwise promote evidence that referred to the correct platform family but not the relevant software package. In such cases, scope-family boosts were

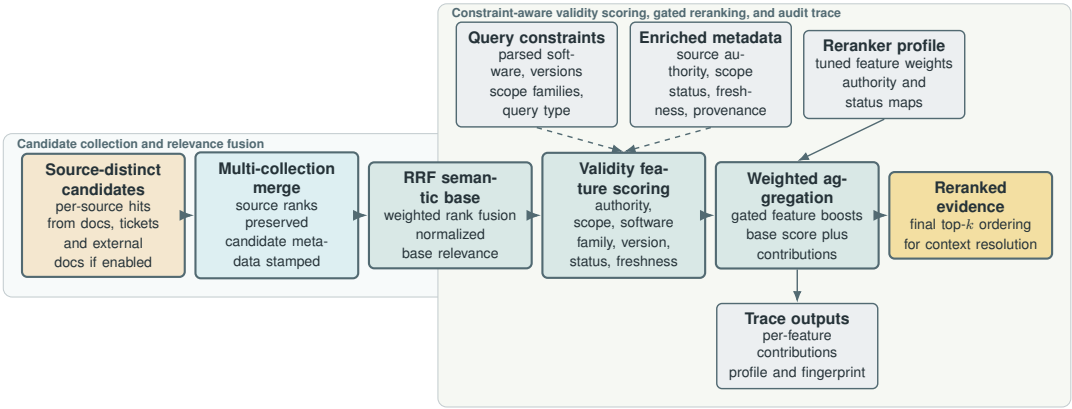

Figure 4.3: Validity-aware reranking in Polaris. Source-distinct candidates are merged and fused into an RRF semantic base, then adjusted using query-aware validity features derived from registry-backed constraints, enriched metadata, and the configured reranker profile before final evidence selection.

gated unless the candidate also satisfied the required software match, and specialised family variants were prevented from receiving an inappropriate generic boost when the exact variant had not actually been requested. On the other hand, for non-software operational queries, scope-family could still provide a useful positive signal.

The final reranker profile was selected on the development split by a two-phase grid search over 47 trials using a coverage-guarded selection rule based on factual-correctness, faithfulness, and context precision. The retained profile achieved development means of 0.314 for factual correctness, 0.613 for faithfulness, and 0.739 for context precision. The active reranker configuration was persisted as a structured profile and fingerprint and logged alongside retrieval traces and final outputs so that ranking behaviour could be reconstructed across runs. Section 5.6.5 describes the full protocol in full.

4.6 Prompt and Generation Design

The generation stage was designed to produce grounded, reviewable support responses. A single fixed prompt template was used within each retrieval comparison so that measured differences could be attributed to retrieval and reranking rather than prompt drift. The final prompt retained a ticket-aligned response schema with fixed sections for classification, assessment, action, customer reply, and safety notes, together with inline numeric citations and a `REFERENCE KEY`. Two few-shot demonstrations were included:

one evidence-sufficient case and one insufficient-evidence case. These demonstrations established the target response structure and modelled conservative behaviour when retrieved evidence was weak [15, 26, 27].

For HPC support, incorrect operational guidance was treated as more harmful than an incomplete answer. When evidence was missing or conflicting, the prompt therefore required explicit uncertainty, the safest next step, and concise follow-up questions rather than unsupported resolution. After generation, post-processing trimmed output drift and rebuilt the `REFERENCE KEY` from citations actually used in the answer body, preventing uncited reference mappings from appearing in the final output.

A later follow-up prompt ablation was conducted after the main error analysis identified generation, rather than retrieval alone, as the remaining performance bottleneck. The refined prompt variant is therefore reported as an additional improvement study rather than as part of the frozen retrieval backbone. The generator configuration used `meta-llama/Llama-3.3-70B-Instruct`, selected in the development experiments, with a 2048-token output limit, temperature 0.8, and top-p 0.9.

4.7 API, Tooling, and Observability

FastAPI was used to expose the system as a typed HTTP service because it provides structured requests and response validation and a stable interface for both the interactive UI and the evaluation tooling [28]. The main API endpoint, `/v1/query`, returns a structured response containing the generated answer, the retrieved contexts, derived query constraints, answer-status metadata, and timing metadata.

The FastAPI requests are governed by a shared request-budget model that allocates a single end-to-end deadline across retrieval and generation. Stage-specific limits are derived from the remaining budget, with an explicit retrieval cap and reserved cleanup margin. This prevents retrieval from exhausting the request window before answer generation completes. The same API also supports optional evaluation metadata, while timeout and internal failures are returned as structured HTTP payloads for downstream inspection.

Operational tooling is exposed through lightweight service endpoints. Basic liveness is separated from dependency-aware readiness by providing both a simple `/health` endpoint and a more complex `/ready` endpoint that checks local runtime dependencies such as the embedding service and the required Qdrant collections. The API also records and aggregates structured user feedback.

The system also records MLflow trace spans across both API and pipeline stages, with correlation headers linking evaluation requests to runtime traces [29]. These traces capture prompts, retrieved contexts, query constraints, timings, and retriever or reranker profiles, making failures easier to debug and experimental runs easier to audit.

4.8 User Interface and Interaction Design

The frontend was implemented as a React single-page application with separate assistant, evaluation, and system views. This separation was chosen so that interactive querying, structured scenario evaluation, and runtime inspection could support different tasks without overloading a single screen. The assistant view opens with both free-text input and predefined scenario prompt cards, allowing either open-ended use or rapid demonstration of representative support queries. After submission, the workspace presents the latest answer alongside a diagnostics panel rather than mixing evidence with the main response. This split keeps the primary support answer readable while still allowing timings, interpreted query constraints, retrieved evidence chunks, and optional debug metadata to be inspected when needed.

As shown in Figure **??**, the evaluation view reuses the same answer and diagnostics components but wraps them in predefined scenarios and a structured feedback form. This allows scenario runs to be compared under a consistent interface while capturing helpfulness, groundedness, citation quality, failure type, and free-text notes. Response fingerprinting is used to prevent duplicate in-session submissions for the same output, making frontend feedback directly reusable in later analysis. A separate system view exposes health, readiness, and runtime configuration so that operational state can be inspected without interfering with either querying or evaluation.

4.9 Verification and Software Testing

This section describes the verification strategy used to check the implementation prior to formal evaluation. The codebase contained 394 named pytest test functions across 53 test modules. The suite was organised by verification layer rather than by package alone so that the highest-risk behaviours could be isolated and regression-tested before benchmark execution. Verification was risk-driven, with priority given to behaviours

most likely to affect answer correctness, runtime robustness, and reviewability. Table 4.2 summarises the main verification layers and the purpose of each.

Table 4.2: Summary of the verification layers used in Polaris and the main behaviours checked at each level.

Layer	Representative checks	Purpose
Unit / component	Authority-registry determinism and candidate extraction; prompt builder rendering; validity-reranker feature scoring, gating, and trace stamping	Verify correctness of core logic in isolation
Pipeline / API	Reference-key sanitisation; anonymisation; answer-status and timing payloads; timeout mapping to structured HTTP failures	Verify end-to-end runtime behaviour and response contracts
Operational / integration	Readiness and tracing behaviour; prompt-registry integration; evaluation-facing API paths and metadata handling	Reduce regression risk across deployed and evaluated system paths

At the component level, tests target the project's highest-risk logic directly. Authority-registry tests checked deterministic artifact construction, candidate extraction, and stable metadata enrichment. Query-constraint tests checked alias resolution, scope-family inference, and version-sensitive parsing. Ticket-chunking tests verified token-budget enforcement, overlap handling, and the three-turn chunk limit. Validity-reranker tests verified official-match promotion, software-gated scope-family boosts, and rerank trace stamping. Pipeline and API tests checked answer-status and timing payloads, reference-key sanitisation, evaluation metadata handling, and the mapping of timeout failures to structured 504 responses. Additional operational tests covered readiness checks, tracing behaviour, prompt-registry integration, and evaluation-facing API paths. This suite did not establish answer quality, which was handled separately by the held-out benchmark, but it did reduce the risk that later empirical findings were artefacts of broken retrieval, prompting, or API plumbing.

4.10 Design Trade-offs

The main drawback of this design was that it introduced a large number of hand-tuned controls. Preserving source distinctions, maintaining the authority registry, tuning chunk settings, and tuning reranker weights improved auditability, but they also

introduced maintenance burden and additional failure modes. Stale registry entries could lead to incorrect metadata enrichment and the hand-tuned reranker weights could overfit the development split. For this reason, these controls were treated as empirical design choices to be tested later rather than as assumptions of automatic improvement.

A similar trade-off appeared at runtime. Structured prompts, reference keys, diagnostics, tracing, and request-budget controls reduced flexibility and added overhead compared with a lightweight question-answering interface. However, these constraints made answers easier to inspect, debug, and evaluate, which was more important for this project than maximising conversational freedom. This trade-off was justified for a supervised support assistant, but it also meant that development effort was spent on control, auditability, and evaluation support rather than on a simpler end-user experience.

Chapter 5

Experimental Methodology

5.1 Evaluation Goals

5.2 Development and Test Split Discipline

5.3 Benchmark Construction and Characterisation

5.4 Frozen Backbone Decisions

5.5 Metric Hierarchy and Decision Rules

5.6 Development-Phase Tuning Protocol

5.6.1 Generator Selection

5.6.2 Chunking Tuning

5.6.3 Retrieval-Mode Pilot

5.6.4 Shared Retrieval Budget Tuning

5.6.5 Validity-Reranker Tuning

5.7 Final Test-Set Experiments

5.7.1 Source Ablation

5.7.2 Validity-Signal Ablation

5.7.3 External-Document Scope Ablation

5.7.4 Tuned Final-Set Comparison

Chapter 6

Results

6.1 Development-Phase Selection Results

6.2 Overview of Main Test-Set Results

6.3 Controlled Ablation Results

6.3.1 Source Ablation

6.3.2 Validity-Signal Ablation

6.3.3 External-Document Scope Ablation

6.4 Practical Comparison Results

6.4.1 Tuned-Finalist Comparison

6.5 Manual Evaluation Results

6.6 Error Analysis Results

6.7 Stability and Robustness Results

Chapter 7

Discussion and Critical Appraisal

7.1 Answers to the Research Questions

7.2 What Improved and Why

7.3 Comparison with Prior Work

7.4 Where the System Still Fails

7.5 Threats to Validity

7.6 Review of the Project Plan

Chapter 8

Conclusion and Future Work

8.1 Summary of Contributions

8.2 Main Findings Against Objectives

8.3 Limitations

8.4 Future Work

Bibliography

[1] "Research Computing Services HPC Documentation — CSD3 1.0 documentation," https://docs.hpc.cam.ac.uk/hpc/index.html.

[2] A. Bondapalli, H. Zheng, O. T. Ajayi, M. Keceli, H. Siddabathuni Som, T. Childers, L. Childers, Y. Ghadar, M. Papka, V. Vishwanath, and R. Ge, "Askhpc: A chatbot for high performance computing user support," in *Proceedings of the SC '25 Workshops of the International Conference for High Performance Computing, Networking, Storage and Analysis*, ser. SC Workshops '25. New York, NY, USA: Association for Computing Machinery, 2025, p. 727–739. [Online]. Available: https://doi.org/10.1145/3731599.3767433

[3] C. Joslin, D. Burns, F. Ashish, and E. Sarbijan, "Generating Frequently Asked Questions from Technical Support Tickets using Large Language Models," in *Proceedings of the SC '25 Workshops of the International Conference for High Performance Computing, Networking, Storage and Analysis*, ser. SC Workshops '25. New York, NY, USA: Association for Computing Machinery, Nov. 2025, pp. 715–726.

[4] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-t. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Proceedings of the 34th International Conference on Neural Information Processing Systems*, ser. NIPS '20. Red Hook, NY, USA: Curran Associates Inc., Dec. 2020, pp. 9459–9474.

[5] Z. Li, J. Zhang, C. Yan, K. Das, S. Kumar, M. Kantarcioglu, and B. A. Malin, "Do You Know What You Are Talking About? Characterizing Query-Knowledge Relevance For Reliable Retrieval Augmented Generation," in *Proceedings of the*

2024 Conference on Empirical Methods in Natural Language Processing, Y. Al-Onaizan, M. Bansal, and Y.-N. Chen, Eds. Miami, Florida, USA: Association for Computational Linguistics, Nov. 2024, pp. 6130–6151.

[6] J. Hwang, J. Park, H. Park, D. Kim, S. Park, and J. Ok, "Retrieval-Augmented Generation with Estimation of Source Reliability," in *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, C. Christodoulopoulos, T. Chakraborty, C. Rose, and V. Peng, Eds. Suzhou, China: Association for Computational Linguistics, Nov. 2025, pp. 34 279–34 303.

[7] M. Westphal and W. Brannath, "Improving Model Selection by Employing the Test Data," in *Proceedings of the 36th International Conference on Machine Learning*. PMLR, May 2019, pp. 6747–6756.

[8] S. Es, J. James, L. Espinosa-Anke, and S. Schockaert, "RAGAS: Automated Evaluation of Retrieval Augmented Generation."

[9] M. Gritta, G. Lampouras, and I. Iacobacci, "HumanRankEval: Automatic Evaluation of LMs as Conversational Assistants," in *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, K. Duh, H. Gomez, and S. Bethard, Eds. Mexico City, Mexico: Association for Computational Linguistics, Jun. 2024, pp. 8237–8249.

[10] Z.-Y. Chen, H. Wang, X. Zhang, E. Hu, and Y. Lin, "Beyond the Surface: Measuring Self-Preference in LLM Judgments," in *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, C. Christodoulopoulos, T. Chakraborty, C. Rose, and V. Peng, Eds. Suzhou, China: Association for Computational Linguistics, Nov. 2025, pp. 1653–1672.

[11] W. Yu, L. Wu, Y. Deng, Q. Zeng, R. Mahindru, S. Guven, and M. Jiang, "Technical Question Answering across Tasks and Domains," in *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies: Industry Papers*, Y.-b. Kim, Y. Li, and O. Rambow, Eds. Online: Association for Computational Linguistics, Jun. 2021, pp. 178–186.

[12] V. Castelli, R. Chakravarti, S. Dana, A. Ferritto, R. Florian, M. Franz, D. Garg, D. Khandelwal, S. McCarley, M. McCawley, M. Nasr, L. Pan, C. Pendus, J. Pitrelli, S. Pujar, S. Roukos, A. Sakrajda, A. Sil, R. Uceda-Sosa, T. Ward, and R. Zhang, "The TechQA Dataset," in *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics*, D. Jurafsky, J. Chai, N. Schluter, and J. Tetreault, Eds. Online: Association for Computational Linguistics, Jul. 2020, pp. 1269–1278.

[13] S. Rodenbeck and Y. Suk Uhr, "TicketHub: Enabling Actionable Analysis of Support Requests With NLP," in *Practice and Experience in Advanced Research Computing 2023: Computing for the Common Good*, ser. PEARC '23. New York, NY, USA: Association for Computing Machinery, Sep. 2023, pp. 109–116.

[14] K. Shuster, S. Poff, M. Chen, D. Kiela, and J. Weston, "Retrieval Augmentation Reduces Hallucination in Conversation," in *Findings of the Association for Computational Linguistics: EMNLP 2021*, M.-F. Moens, X. Huang, L. Specia, and S. W.-t. Yih, Eds. Punta Cana, Dominican Republic: Association for Computational Linguistics, Nov. 2021, pp. 3784–3803.

[15] A. Asai, Z. Wu, Y. Wang, A. Sil, and H. Hajishirzi, "Self-RAG: Learning to Retrieve, Generate, and Critique through Self-Reflection," Oct. 2023.

[16] Z. Xu, M. J. Cruz, M. Guevara, T. Wang, M. Deshpande, X. Wang, and Z. Li, "Retrieval-Augmented Generation with Knowledge Graphs for Customer Service Question Answering," in *Proceedings of the 47th International ACM SIGIR Conference on Research and Development in Information Retrieval*. Washington DC USA: ACM, Jul. 2024, pp. 2905–2909.

[17] J. Ouyang, T. Pan, M. Cheng, R. Yan, Y. Luo, J. Lin, and Q. Liu, "HoH: A Dynamic Benchmark for Evaluating the Impact of Outdated Information on Retrieval-Augmented Generation," in *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, W. Che, J. Nabende, E. Shutova, and M. T. Pilehvar, Eds. Vienna, Austria: Association for Computational Linguistics, Jul. 2025, pp. 6036–6063.

[18] "CSD3 Upgrade October 2021 — CSD3 1.0 documentation," https://docs.hpc.cam.ac.uk/hpc/user-guide/upgrade2021.html.

[19] D. Ru, L. Qiu, X. Hu, T. Zhang, P. Shi, S. Chang, C. Jiayang, C. Wang, S. Sun, H. Li, Z. Zhang, B. Wang, J. Jiang, T. He, Z. Wang, P. Liu, Y. Zhang, and Z. Zhang, "RAGChecker: A Fine-grained Framework for Diagnosing Retrieval-Augmented Generation," in *The Thirty-eight Conference on Neural Information Processing Systems Datasets and Benchmarks Track*, Nov. 2024.

[20] J. Saad-Falcon, O. Khattab, C. Potts, and M. Zaharia, "ARES: An Automated Evaluation Framework for Retrieval-Augmented Generation Systems," in *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies (Volume 1: Long Papers)*, K. Duh, H. Gomez, and S. Bethard, Eds. Mexico City, Mexico: Association for Computational Linguistics, Jun. 2024, pp. 338–354.

[21] "Benchmarking Large Language Models in Retrieval-Augmented Generation | Proceedings of the AAAI Conference on Artificial Intelligence," https://ojs.aaai.org/index.php/AAAI/article/view/29728.

[22] M. Sanderson, "Test Collection Based Evaluation of Information Retrieval Systems," *Foundations and Trends in Information Retrieval*, vol. 4, pp. 247–375, Jun. 2010.

[23] "Collections - Qdrant," https://qdrant.tech/documentation/manage-data/collections/.

[24] Z. Wang, C. Gao, C. Xiao, Y. Huang, S. Si, K. Luo, Y. Bai, W. Li, T. Duan, C. Lv, G. Lu, G. Chen, F. Qi, and M. Sun, "Document Segmentation Matters for Retrieval-Augmented Generation," in *Findings of the Association for Computational Linguistics: ACL 2025*, W. Che, J. Nabende, E. Shutova, and M. T. Pilehvar, Eds. Vienna, Austria: Association for Computational Linguistics, Jul. 2025, pp. 8063–8075.

[25] G. V. Cormack, C. L. A. Clarke, and S. Buettcher, "Reciprocal rank fusion outperforms condorcet and individual rank learning methods," in *Proceedings of the 32nd International ACM SIGIR Conference on Research and Development in Information Retrieval*, ser. SIGIR '09. New York, NY, USA: Association for Computing Machinery, Jul. 2009, pp. 758–759.

[26] T. Brown, B. Mann, N. Ryder, M. Subbiah, J. D. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell, S. Agarwal, A. Herbert-Voss,

G. Krueger, T. Henighan, R. Child, A. Ramesh, D. Ziegler, J. Wu, C. Winter, C. Hesse, M. Chen, E. Sigler, M. Litwin, S. Gray, B. Chess, J. Clark, C. Berner, S. McCandlish, A. Radford, I. Sutskever, and D. Amodei, "Language Models are Few-Shot Learners," *Advances in Neural Information Processing Systems*, vol. 33, pp. 1877–1901, 2020.

[27] T. Gao, H. Yen, J. Yu, and D. Chen, "Enabling Large Language Models to Generate Text with Citations," in *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, H. Bouamor, J. Pino, and K. Bali, Eds. Singapore: Association for Computational Linguistics, Dec. 2023, pp. 6465–6488.

[28] "FastAPI," https://fastapi.tiangolo.com/.

[29] "MLflow - Open Source AI Platform for Agents, LLMs & Models," https://mlflow.org/.

Appendix A

Approved External Documentation Sources

This appendix lists the approved external documentation source families included in the frozen external documentation snapshot used for parameter tuning and final evaluation. These sources were selected as stable, official documentation sources relevant to the CSD3 support domain. The list documents the approved source boundaries used for corpus construction.

Table A.1: Approved external documentation source families included in the frozen external documentation snapshot used for parameter tuning and final evaluation.

Source	Approved scope	Support relevance
Slurm (software)	`https://slurm.schedmd.com/`	Scheduler usage, job submission, resource requests, and runtime behaviour.
CUDA (toolchain)	`https://docs.nvidia.com/cuda/`	GPU programming, CUDA runtime, and accelerator-specific software issues.
Intel oneAPI (toolchain)	`https://www.intel.com/content/www/us/en/docs/oneapi/`	Intel compilers, build configuration, optimisation, and environment setup.
Open MPI (toolchain)	`https://docs.open-mpi.org/en/main/`; `https://docs.open-mpi.org/en/v5.0.x/`	MPI execution, communication behaviour, and version-sensitive runtime detail.
PyTorch (software)	`https://docs.pytorch.org/docs/stable/`	Distributed training, GPU use, Python environments, and framework-specific queries.

Continued on next page

Table A.1 continued from previous page

Source	Approved scope	Support relevance
TensorFlow (software)	`https://www.tensorflow.org/api_docs`; `https://www.tensorflow.org/guide`	Framework APIs, environment setup, and model-execution issues.
GROMACS (software)	`https://manual.gromacs.org/documentation/current/`	Molecular simulation usage, command syntax, and software-specific troubleshooting.
LAMMPS (software)	`https://docs.lammps.org/stable/`	Input syntax, package behaviour, and software-specific execution detail.
OpenFOAM (software)	`https://doc.cfd.direct/openfoam/user-guide-v13/`	Solver usage, case setup, and computational fluid dynamics support issues.
OpenMM (software)	`https://docs.openmm.org/latest/userguide/`	GPU-accelerated molecular simulation, software usage, and configuration questions.
Intel MPI (toolchain)	`https://www.intel.com/content/www/us/en/docs/mpi-library/developer-guide-linux/2021-16/`	MPI runtime behaviour, communication libraries, and parallel-execution troubleshooting.
SingularityCE (software)	`https://docs.sylabs.io/guides/latest/user-guide/`	Container execution, bind mounts, and software-environment portability on HPC systems.
Spack (software)	`https://spack.readthedocs.io/en/latest/`	Package installation, environment composition, and reproducible build workflows.

The approved scope for each source was controlled through the external source registry by restricting crawls to specified domains and included URL prefixes, while excluding archive, release-note, API, PDF, or otherwise misaligned subtrees where appropriate. This ensures that the external corpus remained focused on stable, official, and support-relevant documentation rather than becoming an unrestricted crawl of general web content.

Appendix B

Benchmark Taxonomy Label Definitions

This appendix defines the multi-label taxonomy used to annotate benchmark queries for later subgroup analysis. Each field captures a distinct property of the query or of the minimum evidence conditions required for a correct answer.

Table B.1: Benchmark taxonomy label definitions and allowed values.

Field	What it captures	Allowed values	Annotation rule

`source_needed`	The minimum evidence class needed for a correct answer	`docs`
		`tickets`
		`both`
			Labels describe the task, not the current retrieval result or the current vector-store contents. Assign the minimum evidence class required for a correct answer.

`docs_scope_needed`	Which class of official documentation is authoritative when documentary evidence is needed	`local_official`
		`external_official`
		`local_and_external`
		`none`
			Use `none` only when `source_needed=tickets`. Queries with `source_needed=docs` or `both` must not use `none`.

`validity_sensitive`	Whether correctness depends on current system, partition, service, workflow, or compatibility truth	`yes` `no`	Mark `yes` when current local operational truth is central to correctness; mark `no` when the answer is sufficiently stable.

Continued on next page

Table B.1 continued from previous page

Field	What it captures	Allowed values	Annotation rule
`attachment_dependent`	Whether visible text alone is sufficient, or whether unseen attachments materially determine correct handling	`yes` `no`	Mark `yes` when screenshots, logs, uploaded documents, or other unseen attachments materially affect the correct first response.

`query_type`	The broad problem family represented by the query	`local_operational`
		`software_version`
		`general_how_to`
			Assign the dominant problem family: local service truth, version-sensitive software or toolchain issues, or stable procedural guidance.

`version_sensitive`	Whether correctness depends materially on a software or toolchain version	`yes` `no`	Mark `yes` when version differences could change the correct guidance or resolution.
`system_scope_required`	Whether correctness depends on a particular system, service, partition, or platform	`yes` `no`	Mark `yes` when the answer would differ materially across systems or execution environments.

Where multiple interpretations were possible, labels were assigned according to the minimum evidence needed for a correct and locally applicable first-response answer.

Appendix C

Authority Metadata Fields

Table C.1 summarises the main metadata fields attached during ingestion. These fields are derived from the authority registry and associated preprocessing steps, and are later used to support retrieval, reranking, provenance tracking, and reviewability.

Table C.1: Authority metadata fields attached during ingestion in Polaris.

Field	Description	Role in the system
`source_authority`	Encodes the authority class of the source, such as local official documentation, external official documentation, or ticket memory.	Supports source-aware retrieval and allows later stages to distinguish evidence by reliability class.
`authority_tier`	Numeric authority level derived from the source authority class.	Provides a compact ranking signal for reranking and diagnostic analysis.
`system_names`	Normalised system entities matched to the record.	Identifies which cluster or platform the evidence refers to.
`partition_names`	Normalised partition entities matched to the record.	Supports queries involving scheduler partitions or node classes.

Field	Description	Role in the system
`service_names`	Normalised service entities matched to the record.	Helps capture operational services such as portals or interfaces.
`scope_family_names`	Higher-level scope groupings inferred from matched entities or text patterns.	Allows broader retrieval and reranking decisions beyond exact entity matches.
`software_names`	Normalised software entities matched to the record.	Identifies the main software packages discussed in the evidence.
`software_versions`	Version mentions associated with matched software entities.	Supports version-sensitive retrieval and reranking.
`module_names`	Matched module identifiers extracted from the content.	Helps interpret module-based software references in tickets and documentation.
`toolchain_names`	Normalised toolchain entities matched to the record.	Distinguishes compiler or build-stack context where relevant.
`toolchain_versions`	Version mentions associated with matched toolchain entities.	Supports toolchain-specific matching and version alignment.
`official_doc_matches`	Structured record of registry entity matches back to official documentation.	Preserves documentary support for matched entities and enables later auditability.
`validity_status`	Aggregated lifecycle status inferred from matched registry entities, such as current, maintenance, legacy, or end-of-life.	Supplies a direct validity signal for reranking and review.
`validity_hint`	Structured diagnostic summary of matched status evidence.	Preserves a richer explanation of validity beyond a single label.

Field	Description	Role in the system
`freshness_hint`	Timestamp-like recency cue derived from available metadata fields.	Supports recency-sensitive ranking and later analysis of stale evidence.
`privacy_flags`	Flags indicating the presence of potentially sensitive patterns such as email addresses, IP addresses, or filesystem paths.	Supports governance and safer handling of retrieved ticket evidence.
`provenance`	Structured provenance record including enrichment version, registry artifact path, extraction version, and matched entity identifiers.	Makes enrichment outputs reproducible, traceable, and reviewable.